

AI SYSTEMS ENGINEERING · WEEK 2

Requirements, Risk, and Designing for Mistakes

You cannot specify "correct." You can specify a rate – and design for the remainder.

FAILURE OF THE WEEK

AUC 0.63

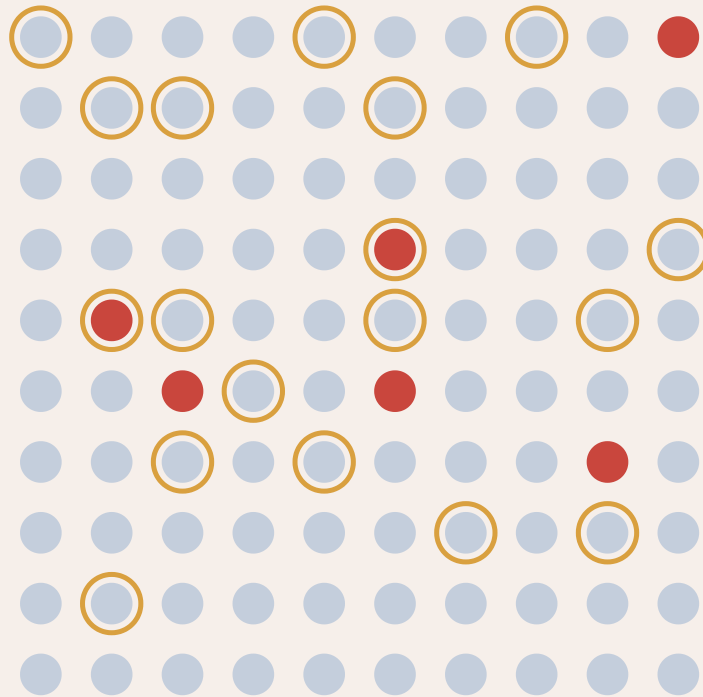
- Sepsis kills; the single biggest survival variable is **how early someone notices**.
- A major EHR vendor shipped a sepsis prediction model to hundreds of hospitals.
- Vendor-reported AUC: **0.76–0.83**.
- In 2021 Michigan Medicine measured it on their own patients: 27,697 patients, 38,455 hospitalizations.

0.63

the AUC they found

◆ *0.5 is a coin. 0.63 is a coin with opinions.*

The operating point



100 hospitalized patients

- a patient
- has sepsis — about 6
- got an alert — 18

Of the 18 alerts, 2 were right.

Of the 6 septic patients, 4 never got an alert.

*Sensitivity 33% · PPV 12% · alert rate 18%
at the vendor-recommended threshold.*

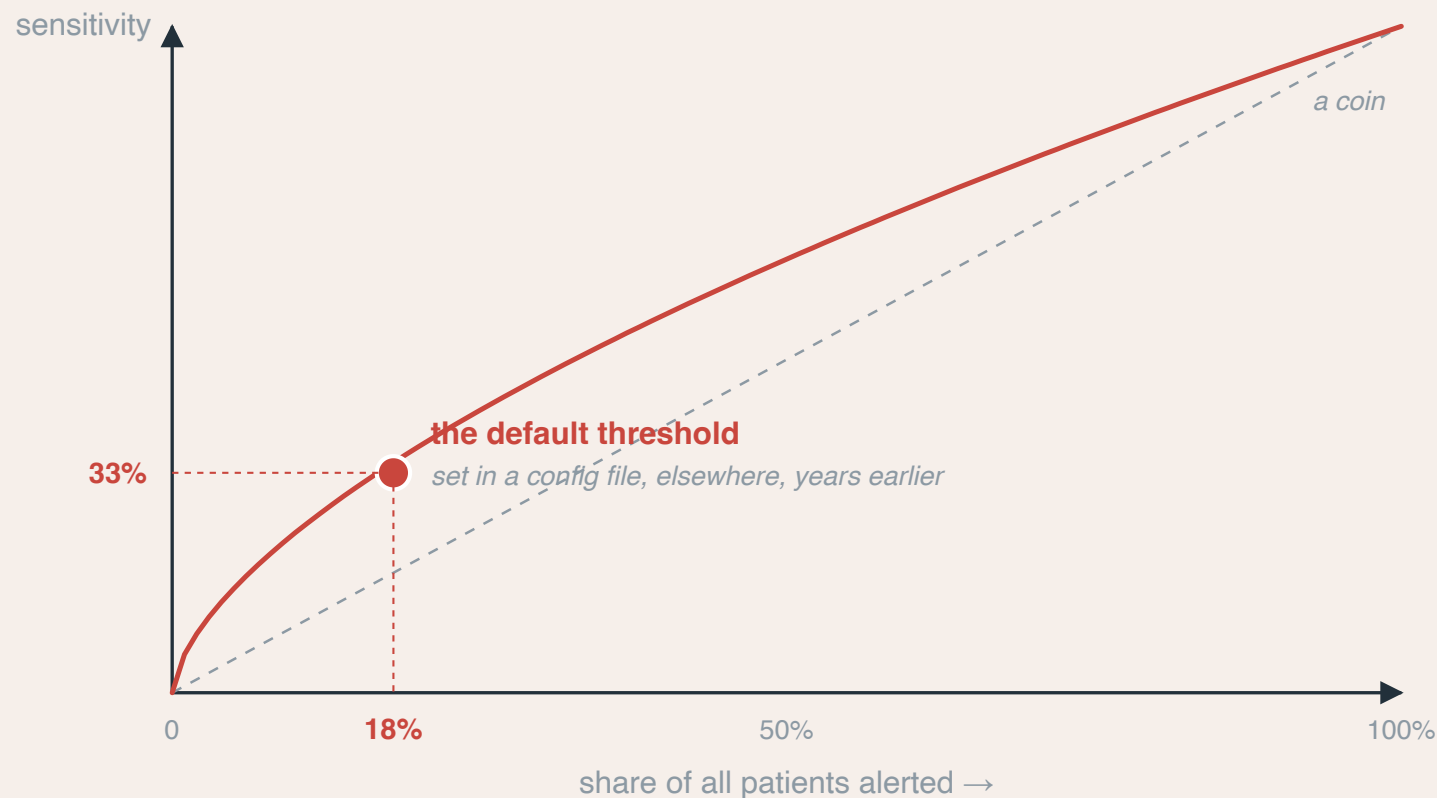
Eight alerts a shift. Seven wrong.

What does a nurse do with that alert after a month?

Wong et al., JAMA Internal Medicine, 2021. Counts rounded to 100 patients.

HOLD THIS THOUGHT

Nobody chose the point



Every point on this curve is a trade: missed cases against wasted attention.

Somebody chooses the point. If nobody does, the default chooses it for them.

The number the hospitals bought was the curve. The system they ran was one point on it.

LECTURE I

The accuracy number is not a requirement

Step 1 is the step everyone skips

Last week: seven steps.

Then the studio. Most of you drew a box within four minutes.

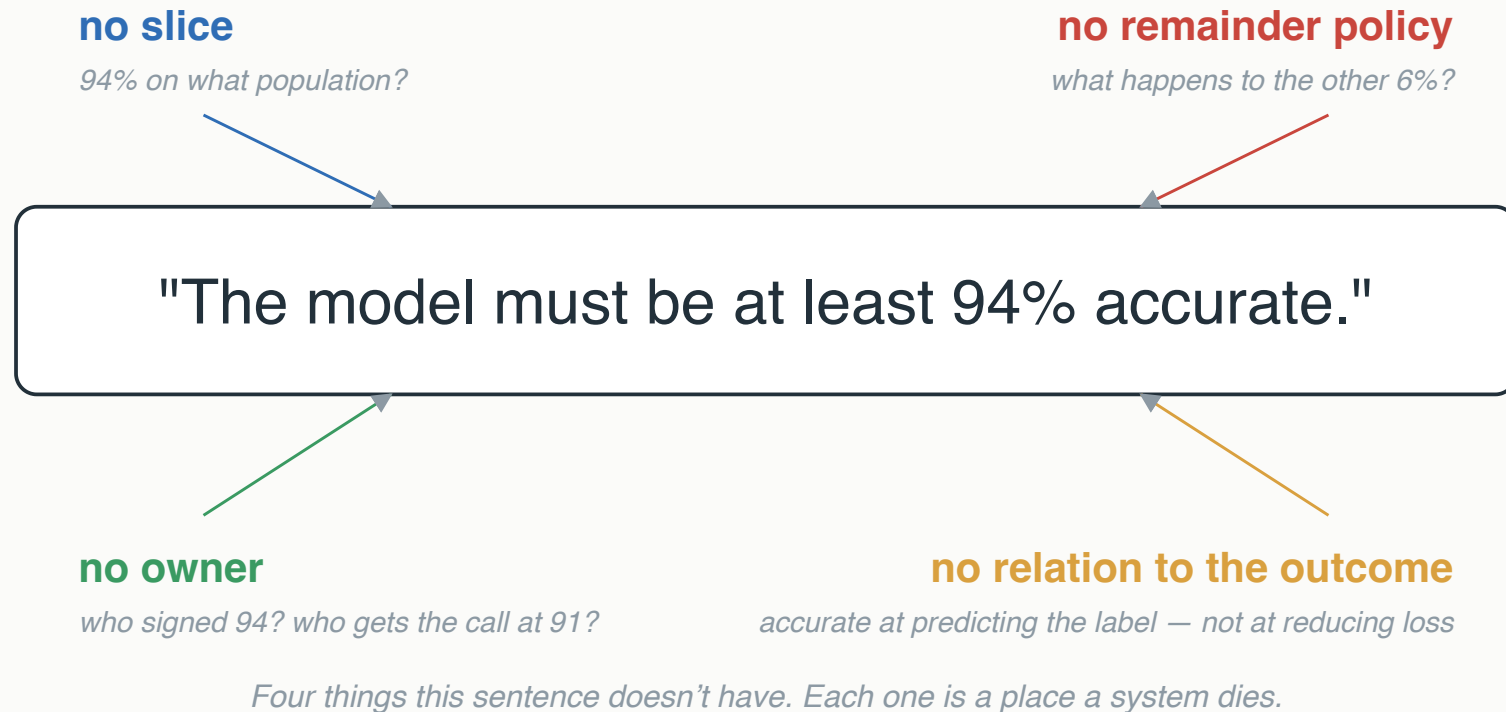
That's why the studio exists.

◆ *I've done this drill with principal engineers. They draw the box faster, because they've had more practice being confident.*

This lecture is steps 1 and 2. Scope and requirements; framing the task.

The least glamorous material in the course, and the material that would have prevented every failure we'll look at this semester.

"94% accurate"



◆ *The AI equivalent of "the server is up." Great. Is it serving anything?*

System goals vs. model goals

Model goals

What the model is optimized for: a loss, a metric, on a distribution.

Owned by the data scientist.

System goals

What the organization is trying to change in the world: fraud loss, resolution time, missed diagnoses, hiring quality.

Owned by somebody with a budget.

Almost every AI project has model goals. Astonishingly few have system goals written in a form you could check. **Requirements is turning the second into constraints on the first — honestly.**

Framing the task: three questions

What decision does the component make? Not "detect fraud." *Assign a score, which a threshold turns into one of three routes: approve, review, block.* The model makes a score. Code makes the route.

What is the I/O contract? Exactly which fields, at what freshness, in — and what out. This is where you discover the feature that only exists *after* the fraud.

Should it be a model at all? The cheapest AI system is the one you replaced with three if-statements. Next week: the determinism boundary.

◆ *The question that gets you fired for asking and promoted for having asked.*

The language of rates

Correct at rate **X** on slice **Y**, with policy **Z** for the remainder, owned by **W**.

X — the rate

a number you can measure

≥ 70% auto-resolved

Y — the slice

the population it applies to

refunds ≤ \$50, account in
good standing

Z — the remainder

the actual path for the failures

escalate with trace; 4h
queue SLO

W — the owner

a name on the org chart

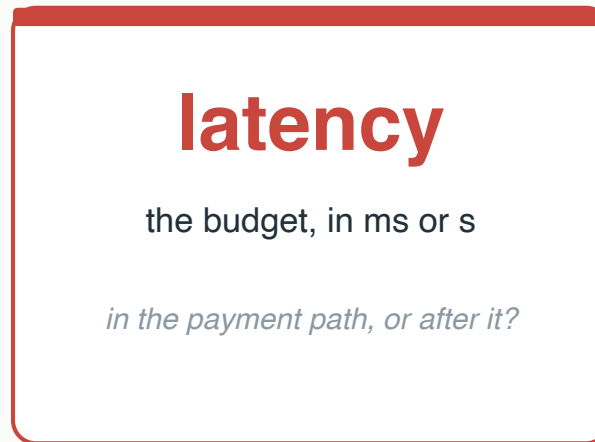
support ops lead

The sentence form. Awkward for two weeks; then you can't write them any other way.

The Operator, rewritten

RATE	SLICE	REMAINDER POLICY	OWNER
Auto-resolves $\geq 70\%$	refunds $\leq \$50$, account in good standing	escalate with trace attached; queue SLO 4h	Support ops lead
Wrong refund amount $\leq 1\%$	all auto-resolved refunds	reversible within 24h; customer notified	Support ops lead
Unauthorized refund = 0	all tickets, incl. adversarial	any occurrence is an incident; agent paused	Security
Escalates $\geq 95\%$	legal threats, safety language, over-limit requests	remainder measured weekly on a labeled sample	Support ops lead
p95 latency ≤ 90 s	all tickets	slow tickets still complete; customer sees progress	Engineering
Cost $\leq \$0.12$ / resolution	monthly average	budget exhaustion pauses auto-resolve; everything escalates	Engineering

Three numbers decide the architecture



Three numbers, in step one. If you don't know them, write down that you don't and who you'll ask.

An architecture designed without them is a guess wearing a diagram.

The trade space

Correctness rate · latency · cost · freshness · robustness · fairness across slices · privacy · explainability
where it binds · safety

You cannot have all of them at the maximum. The requirement *is* the point you choose.

A requirements document that asks for everything has asked for nothing — it hasn't told the engineer which one to give up under pressure.

The conversation nobody has

The acceptable error rate is a **business decision** made with **technical information**.

The data scientist can't make it — they don't own the consequences.

The product owner can't make it alone — they don't know what's achievable.

It gets made in a meeting.

In most organizations the meeting never happens, and the rate gets picked by default —

by whoever set the threshold in a notebook at 2 a.m.

The four questions

1. **At what rate may this be wrong**, on each slice that matters?
2. **What happens to the remainder?** The actual path — not "a human looks at it."
3. **What does a false positive cost, and a false negative?** Dollars, harm, or trust. Never symmetric.
4. **Who owns the residual error?** A name. On the org chart.

In the room:

- Product owner — what the business can absorb
- Data scientist — what's achievable, on which slices
- Engineer — what the remainder path costs to build and run
- Legal / compliance — which numbers are obligations, not preferences

What it costs when nobody does

The default encoded a specific trade — 33% sensitivity for 18% alert volume — that no clinician would have chosen if shown it as a choice.

The choice was made. By whoever set a default in a config file, for a different population, years earlier.

◆ *Every default is a requirement somebody else wrote for you — and didn't sign.*

Regulation as a design input

NIST AI RMF — govern · map · measure · manage.

The seven steps, in a compliance officer's vocabulary.

EU AI Act — unacceptable · high · limited · minimal. High-risk carries obligations: risk management, data governance, logging, human oversight, accuracy you can *show evidence for*.

Two canonical high-risk categories:

- **Employment** — résumé screening. Tonight's studio.
- **Credit** — the Scorer.

"There's a human in the loop," "we log the decision," "the applicant can contest it" — **requirements with the same standing as latency.**

◆ *Regulation is a requirements document written by people who've seen your system fail before you built it.*

LECTURE II

Designing backward from the mistake

The component will be wrong. Forever.

So the design method runs *backward*: enumerate the ways it's wrong, rate them, choose what the system does about each.

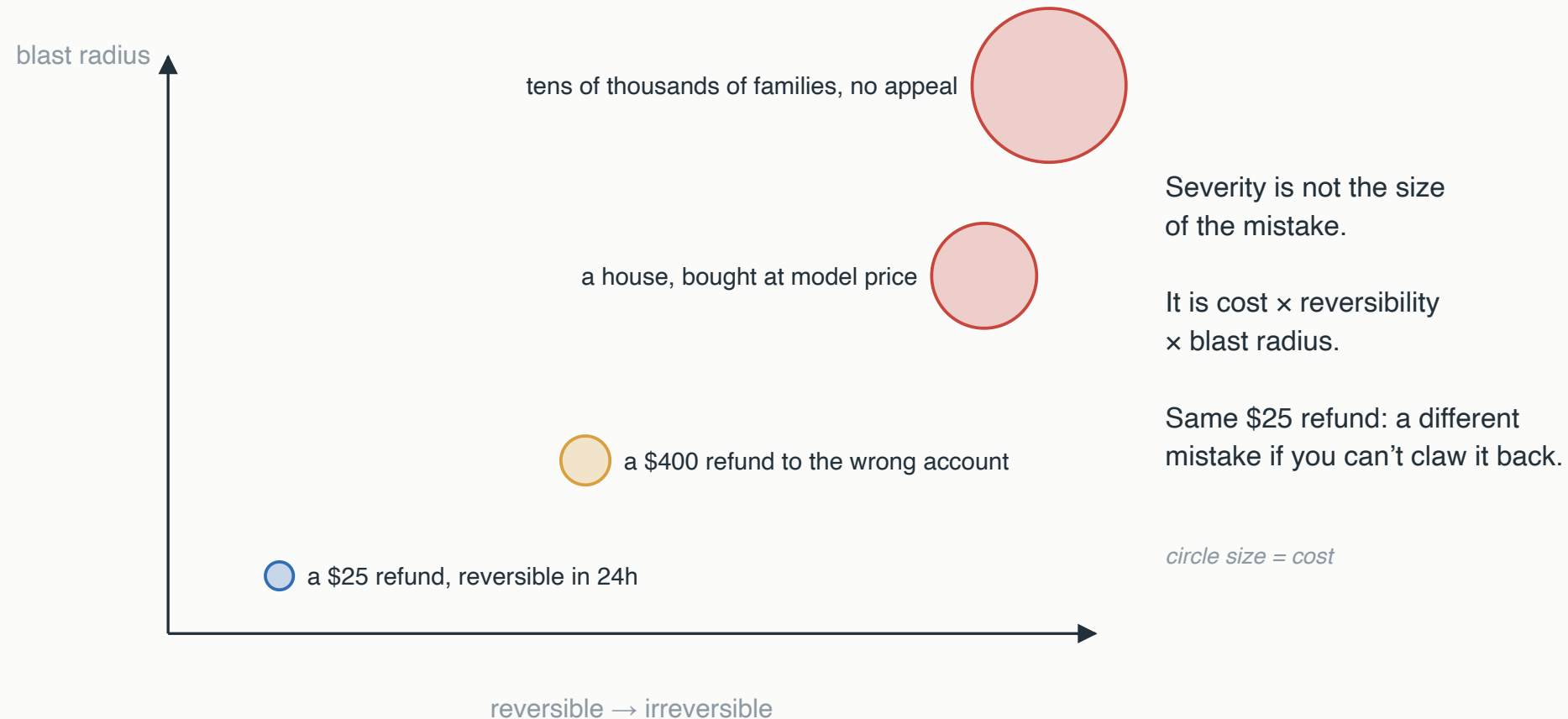
In classic software, error handling is defensive polish.

Here, it is where correctness comes from.

The mistake catalog

KIND	THE OPERATOR'S VERSION
Wrong answer	Refunds \$40 when policy says \$25
Wrong confidence	Sure about the wrong account
Wrong format	Amount in the memo field
Wrong action	Escalates when it should refund — or the reverse
Wrong target	Right amount, wrong customer
No answer	Times out; loops; gives up
Late answer	Correct — after the customer already called
Right answer for the wrong reason	Refunds correctly because <i>the ticket told it to</i>

Severity is not the size of the mistake



Zillow Offers, 2021

The price model estimates what a house is worth. Zillow **buys the house** at that price. With real money.

What does "wrong answer" cost?

Not a number on a screen. **A house. On the balance sheet.** Irreversible. Blast radius: the full purchase price.

And they were moving the comps — feeding their own overestimates back into the data. Property two.

November 2021: shut down. A write-down north of half a billion dollars. A quarter of the company laid off.

"the unpredictability in forecasting home prices far exceeds what we anticipated"

◆ *The model wasn't even bad. It was a good model attached to a checkbook.*

The tools exist

FMEA — failure modes and effects. The mistake catalog with columns. Tonight's lab.

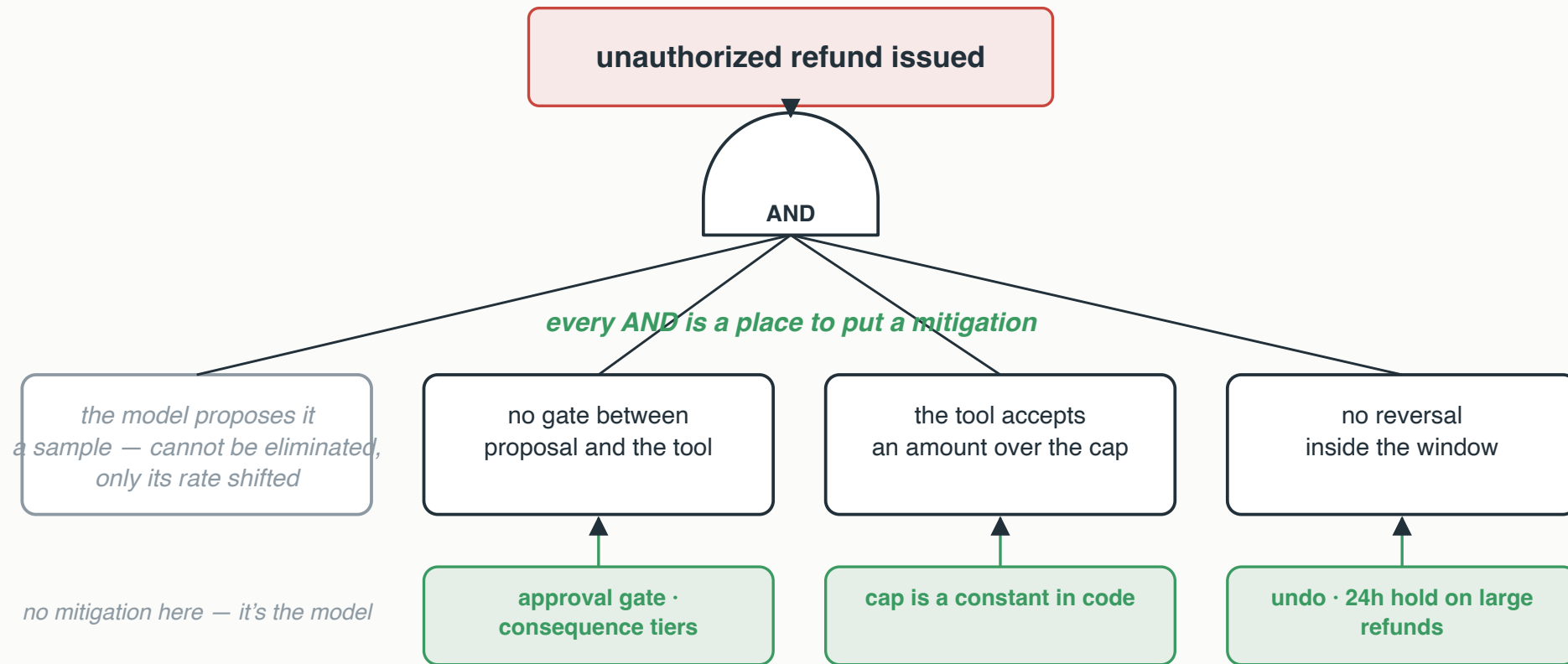
Fault trees — start from the bad outcome, work back through the ANDs. The model being wrong is rarely *sufficient*.

STPA (Leveson) — accidents in complex systems are usually **control failures**, not component failures. Every component did what it was built to do. The control structure was wrong.

The sepsis model did exactly what it was built to do.

The control structure around it — default threshold, alert to a human with no attention budget, no feedback — **was** the failure.

A fault tree for the Operator



The model being wrong is rarely sufficient. Work back from the outcome; the ANDs are your design.

The mitigation toolkit

MITIGATION	WHAT IT COSTS	WHERE IT FAILS
Guardrail	Latency; a second component to evaluate	It's a model — its own error rate
Confidence gate	Coverage	Confidence is a sample; miscalibrated models are confidently wrong
Human in the loop	Throughput; a queue; attention	Automation bias — the human stops looking
Undo / reversibility	Engineering; sometimes impossible	Damage inside the window; things never reversible
Graceful degradation	Quality on the degraded path	Nobody tested the degraded path
Fallback chain	Complexity; every rung needs an eval	Never exercised end to end
Blast-radius scoping	Capability — it can do less	The cap is set by the model instead of code
Rate-limiting the consequence	Throughput	A slow attacker

Two kinds of mitigation

catch the mistake

guardrail	<i>also a model, or also a human</i>
confidence gate	<i>also a model, or also a human</i>
human in the loop	<i>also a model, or also a human</i>
undo / reversibility	<i>also a model, or also a human</i>
graceful degradation	<i>also a model, or also a human</i>
fallback chain	<i>also a model, or also a human</i>

each has an error rate of its own

bound the mistake

amount caps, in code
per-task scope
read-only by default
rate-limit the consequence

**works even when the model
is completely wrong**

Two kinds of mitigation. The second kind is the security half of the course.

Your guardrail is also a model

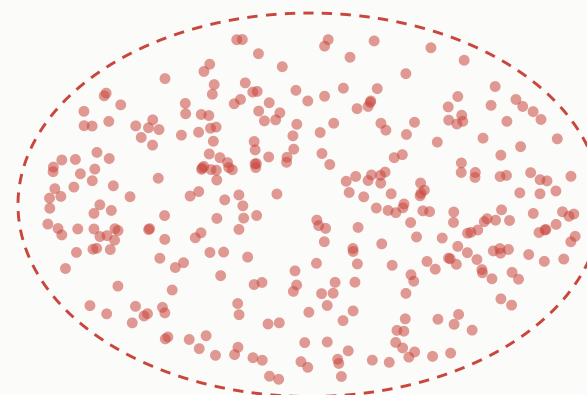
a keyword filter

10 states



what a model can say

effectively unbounded



cannot cover

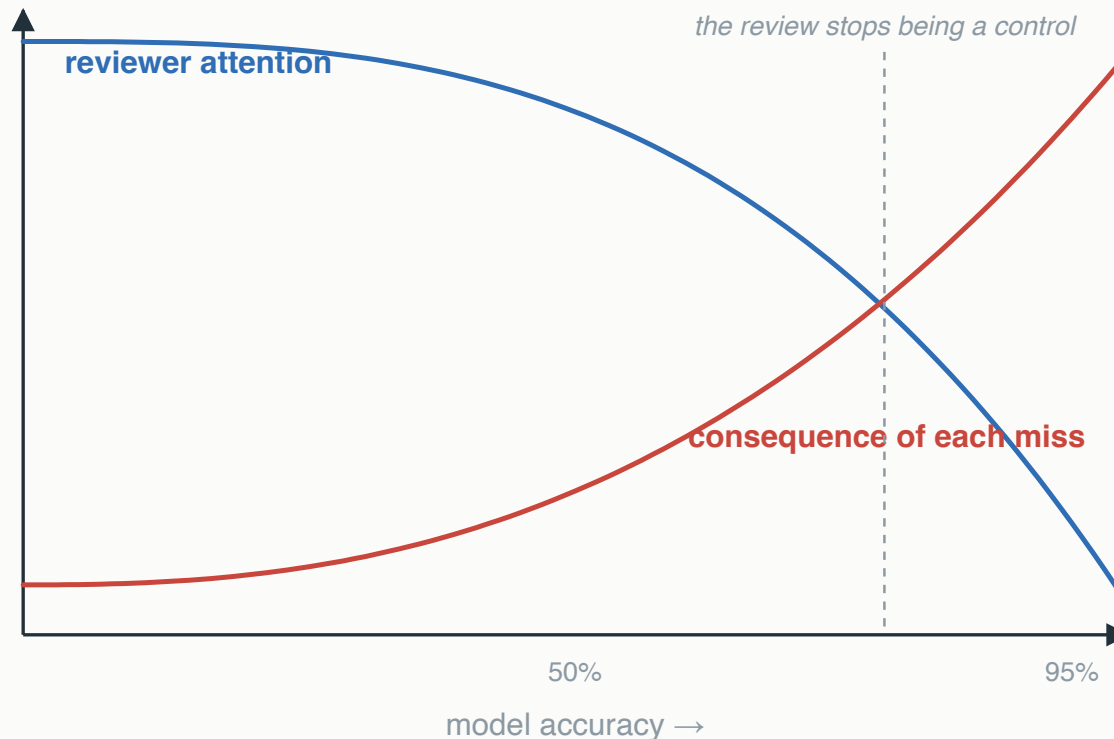
A controller needs at least as much
variety as the thing it controls.

— Ashby, 1956

Guardrails that work are either
models (with everything that implies)
or structural: caps, scopes, permissions,
which never need to understand the output.

◆ *If your guardrail is a regex, your attacker's tool is a thesaurus.*

Automation bias



The better the automation, the worse the human's residual role. — Bainbridge, 1983

At 95% right, the reviewer sees 19 correct outputs per wrong one, learns the system is usually right, and stops reading.

Human-in-the-loop degrades precisely as the model improves. If you propose it, you own this.

Keeping review real

- **Sample, don't gate everything.** 20 hard cases a day get reviewed. 400 easy ones get rubber-stamped.
- **Seed the queue.** Inject known-bad cases. If the reviewer approves one, you've measured the reviewer — and that number goes on the dashboard next to the model's.
- **Rotate.** Attention decays within a session.
- ◆ *"Human in the loop" describes a diagram. "Human who is actually reading" describes a budget.*
- **Give them the trace, not the verdict.** "Approve \$40?" gets a yes. "Here's the ticket, the policy line, and why the agent proposed \$40" gets a read.
- **The human at the moment of consequence, not the moment of connection.** Approve *this* refund, now — not "grant refund permission" once, in September.

The Dutch childcare benefits scandal

Through the 2010s the tax authority ran a risk-scoring system for childcare-benefit fraud. It flagged **tens of thousands of families**. Flags led to repayment demands — often the full amount, years back.

In practice: no way to see why, no effective way to contest. Many flags were wrong. Dual nationality had been used as a risk factor.

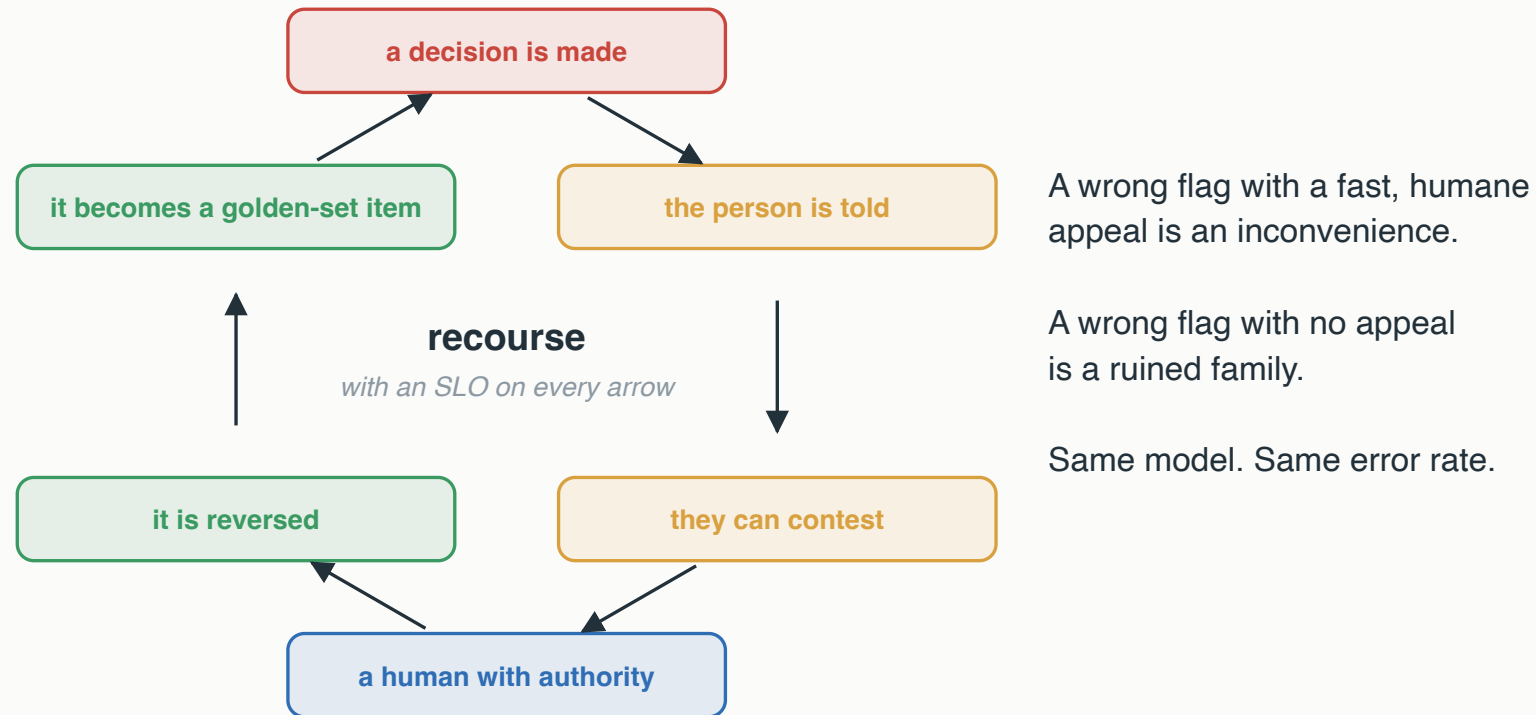
Families lost homes. Children were placed in care.

December 2020: parliamentary report, *Unprecedented Injustice*.

January 2021: the entire cabinet resigned.

The catastrophe was **severity × no recourse**. Same model, same error rate — a different remainder policy is the difference between an inconvenience and a ruined family.

Recourse is a system feature with an SLO



The artifact: the mitigation table

MISTAKE	SLICE	SEVERITY (COST · REVERSIBILITY · RADIUS)	FREQUENCY (EST. → MEASURED)	MITIGATION	ITS FAILURE MODE	OWNER

Three jobs: the requirements for the remainder · the coverage map for your golden set · the seed of P1's **blind-spot register** — the rows where you honestly wrote "we can't detect this."

The last two columns are what separate it from a wish list.

Which step did they skip?

Step 1. No rates, no owner; a vendor's number treated as a requirement.

The meeting. What miss rate is acceptable? What happens to the remainder? A false alert vs. a missed case — priced? Who owns the residual? Every answer was one somebody would have refused to sign. So nobody was asked.

The mitigation was the right one with the predictable failure mode. Bainbridge, 1983. Forty years in the literature.

The model wasn't the problem. A mediocre model with a *chosen* operating point, a priced trade-off, a sampled-and-seeded review queue, and a feedback path would have been a useful system.

The same model with none of those trained clinicians to ignore it.

The remainder is the product.

"A résumé screener rejects a qualified candidate. Redesign the system so that outcome is survivable."

Real category: Amazon built one, scrapped it after it learned to penalize the word "women's."
Under the EU AI Act: **high-risk**.

The trap: "make the model more accurate." The model will be wrong at some rate forever.
Redesign for the remainder.

Build in pairs • then review as a group.

In the review, the four questions, out loud, on the room's design:

Who signs the false-reject rate on the slice "career gap over 12 months"?

♦ *If your answer is "retrain," congratulations: you've designed a system that will reject a different qualified candidate.*

Hazard analysis on the scaffold

1 — Enumerate (20 min). Pairs. README + the three tool descriptions. Using the eight kinds, write **twelve** concrete ways the Operator can be wrong. Concrete: *"refunds the ticket's stated amount instead of the policy amount when they differ."*

2 — Rate (15 min). Severity = cost × reversibility × radius, 1–3 each. Frequency: best guess (P1 measures it). Sort. Top five.

3 — Mitigate (20 min). All seven columns for the top five. **The mitigation's failure mode is mandatory.** If the honest answer is "we can't detect this," write that — it's a blind-spot entry, worth more than a fake guardrail.

Keep: five rows, one with a rate of **zero** and a sentence on why it's an invariant, not a rate. The homework will ask.

This table feeds P1's golden-set coverage and blind-spot register.

Before you go

- **Reading:** book Ch. 8–10 · Huyen, architecture chapters · Dean & Barroso, *The Tail at Scale*
- **P1:** you now have the mitigation table. Golden-set coverage comes from it. Start sourcing tickets this week; the judge comes last.
- Next week: *where does the model live* — and why the best agent is often the one you replaced with a state machine.
- ◆ *Go find out who, at your company, signs the false-positive rate. If the answer is a shrug, you've found your first project.*